



Habib University - City Campus

Course Title: CE/CS 321/330 Computer Architecture

Instructor's name: Dr. Tariq Kamal, Dr. Farhan Khan, Dr. Waseem Hassan

Class ID: 1012 – 1015, 1017

Examination: Final - Term (**SOLUTION**)

Exam Date: 07/05/2024

Total Points: 100

Time/ Duration: 4:30pm - 7:00pm

Instructions for students:

- Any type of smart watch / electronic gadget is prohibited to use during the exam.
- The question paper must be returned at the end of exam along with answer script.
- All answers must be given on answer script.

Guiding note:

“All questions use RISC-V processor and its assembly language. The use of any other processor or its assembly language in answering questions will result in awarding of the lowest possible points by the instructor”

Q1: [LDL-Mapping= Cog6, CLO-3]	(25 points)
Consider the following instruction that is not found in the RISC-V architecture: <pre>Subtract Immediate: subi rd, rs1, Immediate //Reg[rd] = Reg[rs1] - Immediate</pre> We want to modify the RISC-V processor to support the above instruction. For parts (ii) and (iv) below, you can draw your suggested modifications only (no need to draw the entire diagram). For the above instruction, do the following: [5 points] Suggest if any of the existing instruction formats is a good choice to encode the new instruction or what changes in the existing instruction format might be needed. If not, then propose a new instruction format. [5 points] Specify the changes (if any) needed in datapath of the single-cycle RISC-V processor (Figure 1) to execute the new instruction using the instruction format suggested in part i. Use the minimal amount of additional hardware and clock cycles. Remember when adding new instruction, don't break the operation of the standard ones. [3 points] Discuss the effect of the modification in part (ii) on the latency of single-cycle non-pipelined CPU [3 points] Discuss if the suggested modification in part (ii) should be handled by increasing/decreasing the number of pipelining stages that were discussed in class. Draw a pipelined version of the new processor similar to Figure 2 with the needed changes. [5 points] Discuss if any new types of data hazards are introduced due to the new instruction? - If yes, can they be mitigated through forwarding? Use a multi-cycle pipeline diagram to illustrate the new forwarding paths. - If not, specify the TYPES of data hazards which might occur due to this new instruction.	

- vi. **[4 points]** Discuss, based on the above analysis, why the new instruction was not made part of the RISC-V architecture.

Answer:

- i. The format of an I-type instruction can be used to encode subi instruction with the exception that:
- we can now store the Immediate value as an unsigned value increasing the range of numbers so that it now stores numbers from 0 to 4095.
 - A new opcode/funct3 needs to be added for subi instruction.

Immediate	rs1	funct3	rd	opcode
-----------	-----	--------	----	--------

- ii. The basic structure of the processor stays the same. ALU takes the first input from ReadData1 and the second from ImmGen. The computed difference is then passed on to the MUX and then to Writedata in the Register block where the data is written in rd.

The values of the control lines would be as follows (optional):

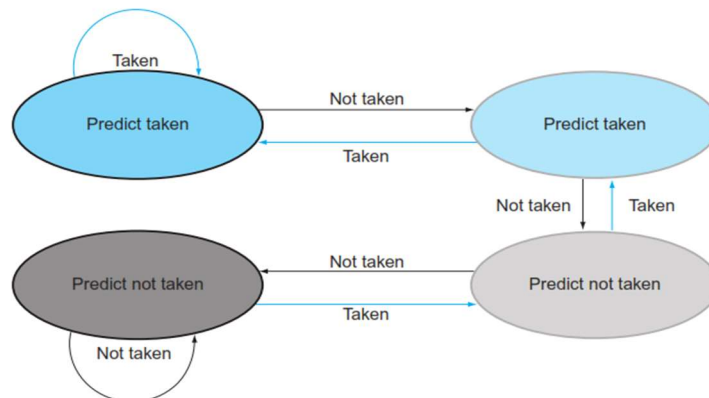
- Branch = 0
 - MemRead = 0
 - MemtoReg = 0
 - ALUOP = 00
 - MemWrite = 0
 - ALUSrc = 1
 - RegWrite = 1
 - ALU Control will need a little modification internally in order to generate control signals for a subtraction operation for subi instead of an addition operation which happens for ld and addi.
- iii. No major components were added only the values for the control signals changed so there would be no change in the latency.
- iv. No change is required in the pipelined version.
- v. No new data hazards would occur. The possible data hazard types are 1a, 1b, 2a, 2b.
- vi. Instead of using subi, just use addi with a negative immediate value, this is what subi would do:
- subi x12, x11, 16 # takes the value from x11, subtracts 16, and stores it in x12

here's what addi does:

addi x12, x11, -16 # takes the value from x11, adds negative 16 (effectively subtracting 16) and stores it in x12.

There's no benefit to having a subi instruction, just use the negative of the immediate value. (The range of the immediate value in addi will be in the range -2048 to 2047).

- a. **[6 points]** Consider three branch prediction schemes: predict not taken, predict taken, and dynamic prediction. Assume that they all have zero penalty when they predict correctly and two cycles when they are wrong. Assume that the average predict accuracy of the dynamic predictor is 90%. Which predictor is the best choice for the following branches?
- A conditional branch that is taken with 5% frequency
 - A conditional branch that is taken with 95% frequency
 - A conditional branch that is taken with 70% frequency
- b. **[19 points]** Consider the following repeating pattern of branch outcomes (e.g., in a loop): T, NT, T, T, NT
- [2 points]** What is the accuracy of always-taken predictor for this sequence of branch outcomes?
 - [2 points]** What is the accuracy of always-not-taken predictor for this sequence of branch outcomes?
 - [5 points]** What is the accuracy of the 2-bit predictor for the first four branches in this pattern, assuming that the predictor starts off in the state “predict not taken”?
 - [5 points]** Design a predictor that would achieve a perfect accuracy if this pattern is repeated forever. Your predictor should be a sequential circuit (e.g., a shift register, or LFSR) with one output that provides a prediction (1 for taken, 0 for not taken) and no inputs other than the clock and the control signal that indicates that the instruction is a conditional branch.
 - [5 points]** Redesign your predictor from part iv but now your predictor should be able to eventually (after a warm-up period during which it can make wrong predictions) start perfectly predicting both this pattern and its opposite. Your predictor should have an input that tells it what the real outcome was. (Hint: this input lets your predictor determine which of the two repeating patterns it is given.)



Answer:

- a) i. Predict not taken ii. Predict Taken iii. Dynamic Prediction

b.

- i. The accuracy of always-taken predictor can be calculated from the following:

PREDICTION	T	T	T	T	T
ACTUAL	T	NT	T	T	NT
OUTCOME	Correct	Incorrect	Correct	Correct	Incorrect

$$\text{Accuracy (T)} = 3/5 = 0.6$$

- ii. The accuracy of always-not-taken predictor can be calculated from the following:

PREDICTION	NT	NT	NT	NT	NT
ACTUAL	T	NT	T	T	NT
OUTCOME	Incorrect	Correct	Incorrect	Incorrect	Correct

$$\text{Accuracy (NT)} = 2/5 = 0.4$$

- iii. The accuracy of 2-bit predictor can be calculated from the following:

PREDICTION	NT	NT	NT	NT
ACTUAL	T	NT	T	T
OUTCOME	Incorrect	Correct	Incorrect	Incorrect

$$\text{Accuracy (DP)} = 1/4 = 0.25$$

- iv. The predictor should be a 5-bit shift register, where 5 is the number of branch outcomes in the target pattern. The shift register should be initialized with the pattern: 1 0 1 1 0 (where 0 for NT, 1 for T), and the prediction is always the value in the left most bit of the shift register. The register should be shifted after each predicted branch.
- v. The predictor is the same as in part iv, except that it should compare its prediction to the actual outcome and invert (logical NOT) all the bits in the shift register if the prediction is incorrect. This predictor still always perfectly predicts the given pattern. For the opposite pattern, the first prediction will be incorrect, so the predictor's state is inverted and after that the predictions are always correct. Overall, there is no warm-up period for the given pattern, and the warm-up period for the opposite pattern is only one branch.

Q3: [LDL-Mapping= Cog4, CLO-4]

(30 points)

a. **[15 points]** Assume we have 32-bit addressing and a direct mapped cache having 8 words per block. Data size for this cache is 128KiB.

- i. **[5 points]** For this cache, calculate the total size (i.e. including tag and valid parts) in bits. Also, find the ratio of total size to the data size.

- ii. **[2 points]** Draw hardware for the cache in part i. Clearly show the length for each of the buses.
 - iii. **[3 points]** If we convert this cache into an 8-way set associative cache, find the length for tag and index fields in bits.
 - iv. **[5 points]** Let's say we have a direct mapped cache with same data size i.e. 128KiB, however the cache is organized in form of double (i.e. 64-bits) instead of words. If the cache has 8 doubles per block, find the bits needed for tag, index and offset.
- b. **[15 points]** Assume a processor is running at 4GHz. The main memory has an access time of 100 nsec. For this processor, a perfect cache (i.e. no misses) has a base CPI is 1.25.
- i. **[5 points]** Find the new CPI for this processor if it has a separate instruction and data cache at first level (L1). Assume an instruction miss rate of 7% and a data miss rate of 5%. In addition, assume that the frequency of data instructions is 40%.
 - ii. **[2 points]** If both instruction and data L1 caches have a hit time of 2.5 nsec, calculate AMATs for both the caches separately.
 - iii. **[8 points]** Let's say we now add separate level two (L2) caches for instruction and data. If both L2 caches have an access time of 10 nsec, whereas instruction miss rate changes to 1% and data miss rate changes to 0.5% after addition of L2 cache, find the new CPI (i.e. with both L1 and L2 caches) and calculate speedup compared to part i.

Answer:

- a) Assume we have 32-bit addressing and a direct mapped cache having 8 words per block. Data size for this cache is 128KiB. **[15 points]**
 - i. For this cache, calculate the total size (i.e. including tag and valid parts) in bits. Also, find the ratio of total size to the data size. **[5 points]**

Sol:

$$2^m = 8 \rightarrow m = 3 \text{ (block offset} \rightarrow 3 \text{)}$$

$$128 \text{ KiB} \rightarrow 32 \text{ KiWords} \rightarrow 4 \text{ KiBlocks} = 4096 \text{ blocks} \rightarrow 2^n = 4096 \rightarrow n = 12 \text{ (index bits)}$$

$$\text{Tag bits} = 32 - n - m - 2 = 32 - 12 - 3 - 2 = 15$$

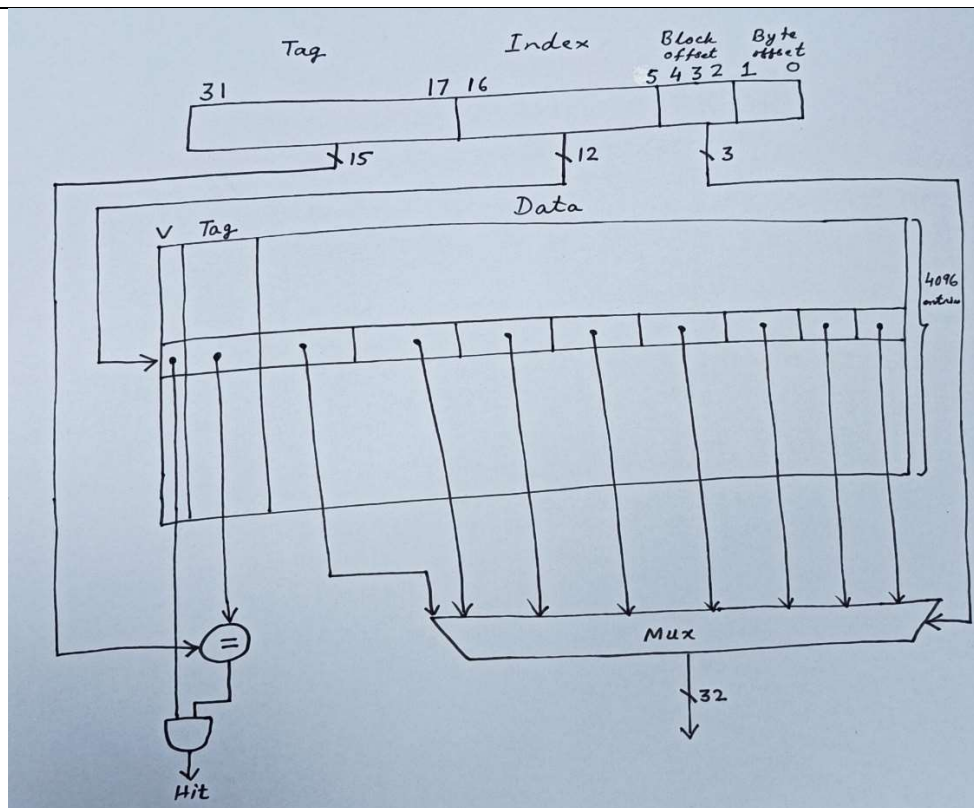
$$\text{Size of a single cache line} = \text{tag bits} + \text{valid bit} + \text{block size} = 15 + 1 + 8 \times 32 = 272 \text{ bits}$$

$$\text{Size of cache} = \text{size of cache line} \times \text{number of lines} = 272 \times 4096 = 1114112 \text{ bits} = 136 \text{ KiBytes}$$

$$\text{Ratio of total size to data size} = 136 \text{ KiBytes} / 128 \text{ KiBytes} = 1.0625$$

- ii. Draw a hardware for the cache in part i. Clearly show the length for each of the buses. **[2 points]**

Sol:



Notes:

1. Address will be 32 bits long
2. Index will be 12 bits long, running from 0 to 4095
3. Tag will be 15 bits long
4. Data will be 8×32 bits long
5. m (block offset) could be used with MUX for selection of a word within a block

iii. If we convert this cache into 8-way set associative cache, find the length for tag and index fields in bits. **[3 points]**

Sol:

Block offset i.e. words per block will remain same.

No. of sets = Total no. of blocks / Blocks per set = $4096/8 = 512$ sets = 2^9

Index $\rightarrow$ 9 bits from 0 till 511

Tag_{new} = Tag_{old} + bits from index = $15 + 3 = 18$ bits

iv. Let's say we have a direct mapped cache with same data size i.e. 128KiB, however the cache is organized in form of double (i.e. 64-bits) instead of words. If the cache has 8 doubles per block, find the bits needed for tag, index and offset. **[5 points]**

Sol:

$2^m = 8 \rightarrow m = 3$ (block offset $\rightarrow 3$)

128 KiB $\rightarrow$ 16 KiDoubles $\rightarrow$ 2 KiBlocks = 2048 blocks $\rightarrow 2^n = 2048 \rightarrow n = 11$ (index bits)

Tag bits = $32 - n - m - 3 = 32 - 11 - 3 - 3 = 15$ (we are using 3 now since double is 8 bytes)

b) Assume a processor is running at 4GHz. The main memory has an access time of 100 nsec. For this processor, a perfect cache (i.e. no misses) has a base CPI is 1.25. **[15 points]**

i. Find the new CPI for this processor if it has a separate instruction and data cache at first level (L1). Assume an instruction miss rate of 7% and a data miss rate of 5%. In addition, assume that the frequency of data instructions is 40%. **[5 points]**

Sol:

$CPI_{new} = CPI_{base} + CPI_{inst} + CPI_{data}$

$miss_penalty = main_memory_time = 100 \text{ nsec} \rightarrow 100 \text{ nsec} / (1/4\text{GHz}) = 400 \text{ clock cycles}$

$CPI_{inst} = I * instr_miss_rate * miss_penalty = I * 7\% * 400 = 28I$

$CPI_{data} = I * data_inst_percent * data_miss_rate * miss_penalty = I * 40\% * 5\% * 400 = 8I$

$CPI_{new} = 1.25 + 28 + 8 = 37.25$ (I could be dropped here)

ii. If both instruction and data L1 caches have a hit time of 2.5 nsec, calculate AMATs for both the caches separately. **[2 points]**

Sol:

$hit_time = 2.5\text{nsec} \rightarrow 2.5 \text{ nsec} / (1/4\text{GHz}) = 10 \text{ clock cycles}$

$inst_AMAT = hit_time + inst_miss_rate * miss_penalty = 10 + 7\% * 400 = 38 \text{ clock cycles}$

$data_AMAT = hit_time + data_miss_rate * miss_penalty = 10 + 5\% * 400 = 30 \text{ clock cycles}$

iii. Let's say we now add separate level two (L2) caches for instruction and data. If both L2 caches have an access time of 10 nsec, whereas instruction miss rate changes to 1% and data miss rate changes to 0.5% after addition of L2 cache, find the new CPI (i.e. with both L1 and L2 caches) and calculate speedup compared to part i. **[8 points]**

Sol:

$CPI_{new} = CPI_{base} + CPI_{instL1L2} + CPI_{dataL1L2}$

$L2_access_time = 10 \text{ nsec} \rightarrow 10 \text{ nsec} / (1/4\text{GHz}) = 40 \text{ clock cycles}$

$$\text{CPI}_{\text{instL1L2}} = I * (7\% * 40 + 1\% * 400) = 6.8I$$

$$\text{CPI}_{\text{dataL1L2}} = I * 0.4 * (5\% * 40 + 0.5\% * 400) = 1.6I$$

$$\text{CPI}_{\text{new}} = 1.25 + 6.8 + 1.6 = 9.65 \text{ (I could be dropped here)}$$

$$\text{speedup} = \text{CPI}_{\text{L1}} / \text{CPI}_{\text{L1L2}} = 37.25 / 9.65 = 3.86$$

Q4: [LDL-Mapping= Cog4, CLO-4]

(20 points)

Let's say we have a array called data[16] stored in memory. Size of array is 16-words as shown in the following table:

Byte address	0	4	8	12	16	...	56	60
Word address	0	1	2	3	4	...	14	15
Array element	data[0]	data[1]	data[2]	data[3]	data[4]	...	data[14]	data[15]

a. [13 points] Consider following address stream:

Word addresses requested by processor: 0, 6, 2, 7, 5, 3, 4, 1, 2, 7, 0, 6

- [3 points] Assume we have a direct mapped cache having one word per block and 8 blocks in total. For each of the given addresses, write the cache index where the data will be placed and state whether the access will result in a miss or hit. Also, calculate miss rate. [Hint: draw a table showing word address, cache index and hit/miss]
- [5 points] Now if we change the block size to two words per block while the cache is kept direct mapped and total number of blocks do not change. Do part i again showing which addresses will result in miss and which will be a hit. Also, calculate miss rate. [Hint: it will decrease the index by a factor of 2 since total cache size is kept constant. Draw again same table]
- [5 points] If we have a 16 bit memory address, calculate tag, index and offset fields for the caches in part i and part ii.

b. [7 points] Let's say we change the address stream:

New word address stream: 0, 5, 2, 7, 15, 10, 13, 8, 7, 0, 2, 13, 4, 1, 14, 11

- [5 points] Assume we have a 2-way set associative cache with one word per block and 8 blocks in total. For each of the given addresses, write the index where the data will be placed and state whether the access will result in a miss or hit. In addition, specify if a blocked will be replaced or not. Use LRU, if needed, as replacement policy. Assume at the start, all valid bits are 0 i.e. cache is empty and does not contain a valid data. Finally, calculate miss rate for this cache. [Hint: draw a table showing word address, cache index, replaced or not and hit/miss]

- ii. **[2 points]** Belady's replacement algorithm, is an optimal cache replacement algorithm that prioritizes discarding/replacing information not needed for the longest time in the future. Despite its optimality compared to other algorithms, it is never implemented in practical caches. Why is this algorithm never used? Justify your answer. Maximum 3 sentences

Answer:

- a) Consider following address stream:

Word addresses requested by processor: 0, 6, 2, 7, 5, 3, 4, 1, 2, 7, 0, 6 **[13 points]**

- i. Assume we have a direct mapped cache having one word per block and 8 blocks in total. For each of the given addresses, write the cache index where the data will be placed and state whether the access will result in a miss or hit. Also, calculate miss rate. [Hint: draw a table showing word address, cache index and hit/miss] **[3 points]**

Sol:

Cache size → 8 words (1 word per block)

Index → 0 to 7

Cache type → Direct-Mapped

Placement formula → (block address) mod (number of blocks)

(block address) mod (8)

Block and word addresses are same since 1 word per block

Word/block address	Hit/Miss	Cache index
0	Miss	data[0] → index 0
6	Miss	data[6] → index 6
2	Miss	data[2] → index 2
7	Miss	data[7] → index 7
5	Miss	data[5] → index 5
3	Miss	data[3] → index 3
4	Miss	data[4] → index 4
1	Miss	data[1] → index 1
2	Hit	Will be read only, index 2
7	Hit	Will be read only, index 7
0	Hit	Will be read only, index 0
6	Hit	Will be read only, index 6

Total accesses = 12; Hits = 4; Misses = 8

Miss rate = $8/12 = 0.666$; Hit rate = $4/12 = 0.333$

- ii. Now if we change the block size to two words per block while the cache is kept direct mapped and total number of blocks do not change. Do part i again showing which addresses will result in miss and which will be a hit. Also, calculate miss rate. [Hint: it will decrease the index by a factor of 2 since total cache size is kept constant. Draw again same table] **[5 points]**

Sol:

Cache size → kept constant i.e. 8 words/blocks from last part

However, arrangement is 2 words per block

Index → 0 to 3 (8/2)

Cache type → Direct-Mapped

Placement formula → (block address) mod (number of blocks)

(block address) mod (8)

Block and word addresses are NOT same anymore

Block address = $\text{floor}(\text{byte address} / \text{bytes per block})$ (See example on page 382)

bytes per block = $4 * 2 = 8$

Byte address	0	4	8	12	16	20	24	28	32	36
Word address	0	1	2	3	4	5	6	7	8	9
Block address	0	0	1	1	2	2	3	3	4	4

Word address	Block address	Hit/Miss	Cache index
0	0	Miss	data[0, 1] → index 0
6	3	Miss	data[6, 7] → index 3
2	1	Miss	data[2, 3] → index 1
7	3	Hit	Will read index 3
5	2	Miss	data[4, 5] → index 2
3	1	Hit	Will read index 1
4	2	Hit	Will read index 2
1	0	Hit	Will read index 0
2	1	Hit	Will read index 1
7	3	Hit	Will read index 3
0	0	Hit	Will read index 0
6	3	Hit	Will read index 3

Total accesses = 12; Hits = 8; Misses = 4

Miss rate = $4/12 = 0.333$; Hit rate = $8/12 = 0.666$

- iii. If we have a 16 bit memory address, calculate tag, index and offset fields for the caches in part i and part ii. **[5 points]**

Sol:

Address = 16 bit

part i: 8 index values $\rightarrow$ 3 bit index ($8 = 2^3$)

$$2^m = 1 \text{ (words per block)} \rightarrow m = 0$$

byte offset = 2 since cache is organized as words (4 bytes)

$$\text{Tag} = 16 - m - n - 2 = 16 - 0 - 3 - 2 = 11 \text{ bits}$$

part ii: 4 index values $\rightarrow$ 2 bit index ($4 = 2^2$)

$$2^m = 2 \text{ (words per block)} \rightarrow m = 1$$

byte offset = 2 since cache is organized as words (4 bytes)

$$\text{Tag} = 16 - m - n - 2 = 16 - 1 - 2 - 2 = 11 \text{ bits}$$

b) Let's say we change the address stream:

New word address stream: 0, 5, 2, 7, 15, 10, 13, 8, 7, 0, 2, 13, 4, 1, 14, 11 **[7 points]**

- i. Assume we have a 2-way set associative cache with one word per block and 8 blocks in total. For each of the given addresses, write the index where the data will be placed and state whether the access will result in a miss or hit. In addition, specify if a blocked will be replaced or not. Use LRU, if needed, as replacement policy. Assume at the start, all valid bits are 0 i.e. cache is empty and does not contain a valid data. Finally, calculate miss rate for this cache. [Hint: draw a table showing word address, cache index, replaced or not and hit/miss] **[5 points]**

Sol:

2-way set associative $\rightarrow$ 2 blocks in each set

1 word per block $\rightarrow$ each set will have 2 words in a single cache set

8 blocks in total $\rightarrow$ total 4 cache sets (8/associativity)

2-way set associative, 1 word per block

Possible word addresses	Set Index	Block 0	Block 1
0, 4, 8, 12	0	1 word	1 word
1, 5, 9, 13	1	1 word	1 word
2, 6, 10, 14	2	1 word	1 word
3, 7, 11, 15	3	1 word	1 word

Word address	Set index	Hit/Miss	Explanation
0	0	Miss	Stored in set 0 (could be in block 0/1)
5	1	Miss	Stored in set 1 (could be in block 0/1)
2	2	Miss	Stored in set 2 (could be in block 0/1)
7	3	Miss	Stored in set 3 (could be in block 0/1)
15	3	Miss	Stored in set 3
10	2	Miss	Stored in set 2
13	1	Miss	Stored in set 1
8	0	Miss	Stored in set 0
7	3	Hit	Read set 3
0	0	Hit	Read set 0
2	2	Hit	Read set 2
13	1	Hit	Read set 1
4	0	Miss	Replace set 0 block 1 (LRU)
1	1	Miss	Replace set 1 block 0 (LRU)
14	2	Miss	Replace set 2 block 1 (LRU)
11	3	Miss	Replace set 3 block 1 (LRU)

Total accesses = 16; Hits = 4; Misses = 12
Miss rate = $12/16 = 0.75$; Hit rate = $4/16 = 0.25$

- ii. Belady's replacement algorithm, is an optimal cache replacement algorithm that prioritizes discarding/replacing information not needed for the longest time in the future. Despite its optimality compared to other algorithms, it is never implemented in practical caches. Why is this algorithm never used? Justify your answer . Maximum 3 sentences **[2 points]**

Sol:

Knowing future accesses is not possible. Therefore, we cannot implement a hardware which predicts future (accesses).

CACHE MEMORY FORMULAS
For a direct-mapped cache with 64-bit address bus: • The cache size is 2^n blocks, so n bits are used for the index • The block size is 2^m words (2^{m+2} bytes), so m bits are used for the word within the block, and two bits are used for the byte part of the address • The size of the tag field is $64 - (n + m + 2)$ • The total number of bits in a direct-mapped cache is $2^n \times (\text{block size} + \text{tag size} + \text{valid field size})$
AMAT = Time for a hit + Miss rate $\times$ Miss penalty

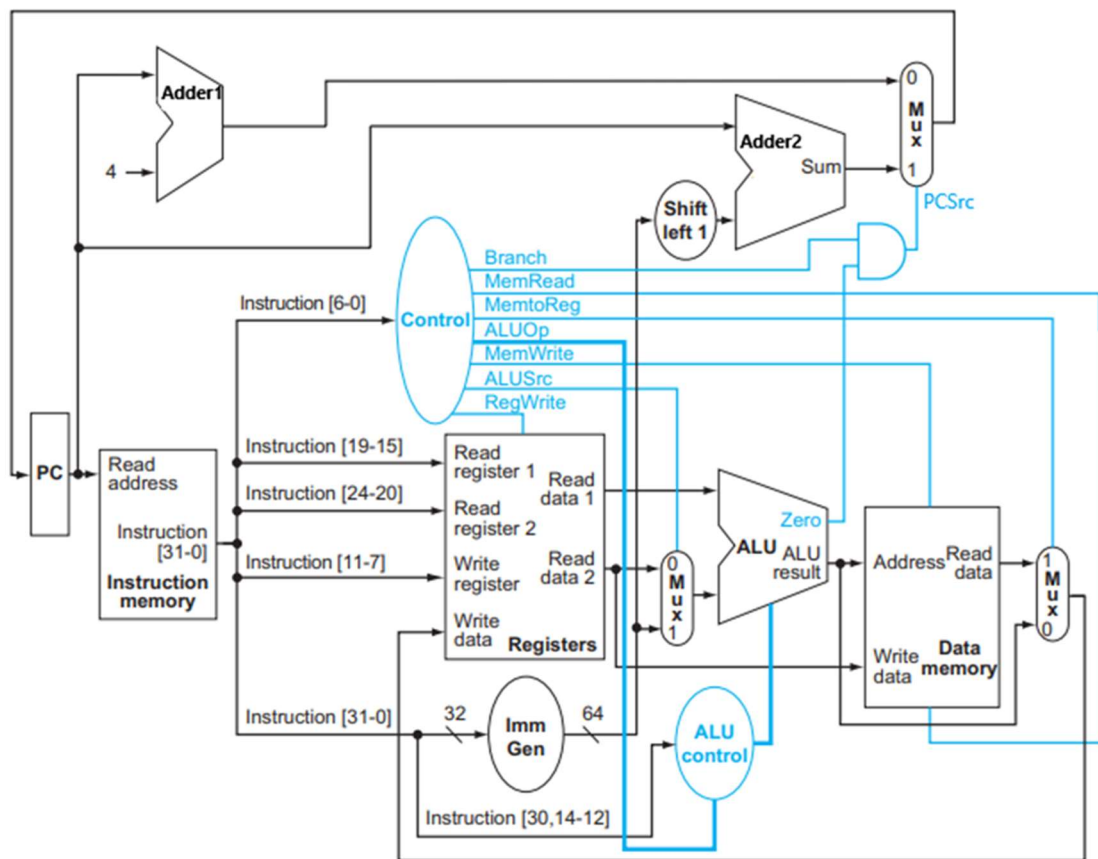


Figure 1.

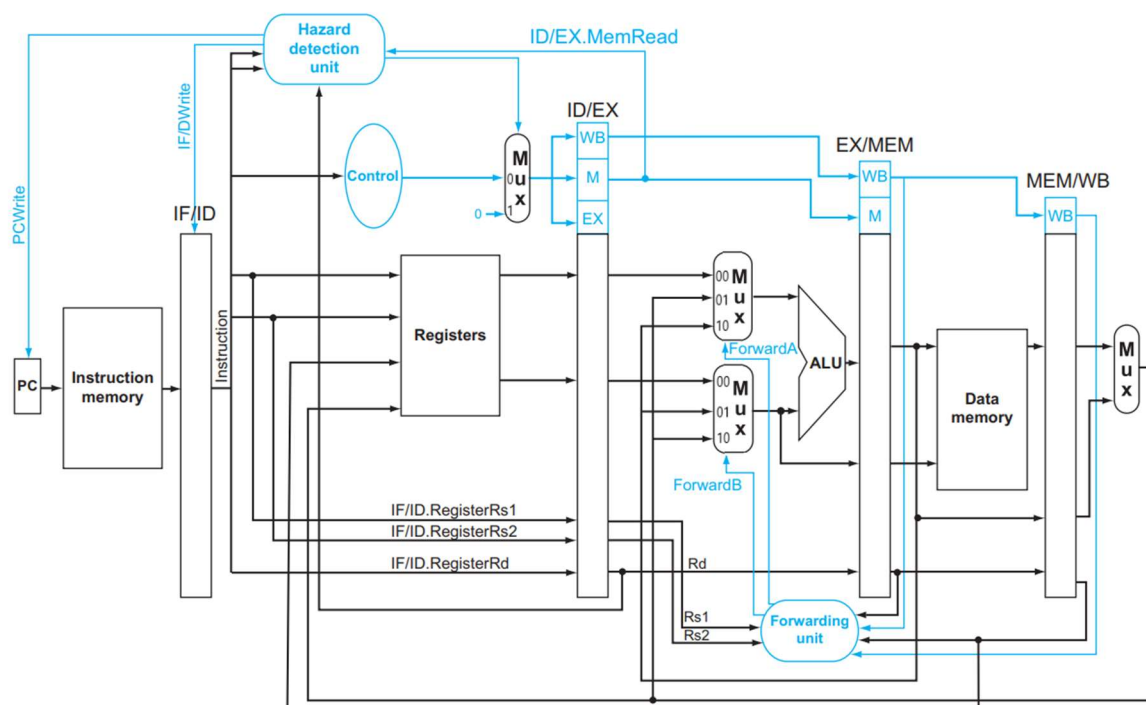


Figure 2.